

Retrieval-Augmented Prediction for Tabular Foundation Models under Heavy Class Imbalance

Test-time context construction for in-context-learning fraud detection

Theo

2026-06-13

Abstract Financial fraud detection is an extreme imbalanced-learning problem: fraudulent records form a tiny fraction of all activity, yet the cost of missing them is high. Tabular foundation models (FTMs) such as the TabPFN family and TabICLv2 have recently matched or surpassed gradient-boosted decision trees on tabular classification through in-context learning, predicting by conditioning on a training context at inference time rather than by training a model per dataset. These models, however, have a fixed and modest context window, and under heavy class imbalance the standard practice of filling it by uniform random subsampling wastes almost the entire window on redundant legitimate records while admitting only a handful of fraud examples. This thesis proposes Retrieval-Augmented Prediction (RAP), a test-time context-construction strategy that, for each batch of records to be scored, retrieves a structurally similar context by k-nearest-neighbour search and rebalances it to a controlled fraud ratio by oversampling. RAP operates entirely at inference on a frozen model and requires no retraining. We evaluate five FTMs and two gradient-boosted baselines on six public fraud-detection datasets, using prevalence-corrected PR-AUC and recall at fixed false-positive rates, and decompose RAP’s benefit into a relevance component (retrieval) and a ratio component (oversampling).

Table of contents

1	Introduction	3
1.1	Fraud detection is a hard imbalanced-learning problem	3
1.2	Gradient-boosted trees and the rise of tabular foundation models	3
1.3	The problem: a fixed context window wasted under imbalance	4
1.4	The idea: Retrieval-Augmented Prediction	4
1.5	Contributions	5

1.6	Research questions	5
1.7	Thesis outline	6
2	Background	7
2.1	Supervised fraud detection under extreme class imbalance	7
2.1.1	Evaluation metrics	7
2.1.2	Classic imbalance remedies	8
2.2	Tabular machine learning and gradient-boosted decision trees	8
2.3	In-context learning and Prior-Data Fitted Networks	9
2.4	Tabular foundation models	9
2.5	The context-window bottleneck under imbalance	10
3	Related Work	12
3.1	The TabPFN lineage and the scaling of context	12
3.2	Foundation models and transformers in fraud and finance	13
3.3	Classical and deep fraud detection and class imbalance	13
3.4	Retrieval-augmented and context-selection for in-context learning	14
3.5	Summary of gaps	15
4	Methods	16
4.1	Problem formulation	16
4.2	The RAP pipeline	17
4.3	Experimental conditions	18
4.4	Models	19
4.5	Evaluation metrics	20
5	Experimental Setup	23
5.1	Datasets	23
5.2	Splitting protocol	24
5.3	Preprocessing	25
5.4	Implementation and reproducibility	26
5.5	Compute environment	26
5.6	The experiment grid	27
6	Results, Discussion and Conclusion	28
6.1	Status of the experimental grid	28
6.2	RQ1 — FTMs versus GBDTs under random subsampling (C1)	29
6.3	RQ2 — does random subsampling degrade at scale? (C1 across size tiers)	29
6.4	RQ3 — does retrieval improve over random subsampling? (C1 $\rightarrow$ C3 $\rightarrow$ C4 ablation)	30
6.5	RQ4 — the optimal context fraud ratio (C4 sweep)	31
6.6	Discussion	31
6.7	Conclusion, Limitations, and Future Work	32
6.7.1	Limitations	32
6.7.2	Future work	33
6.7.3	Restatement of the contribution	33
	References	35
	Bibliography	35

1 Introduction

1.1 Fraud detection is a hard imbalanced-learning problem

Financial fraud detection is the task of flagging the small fraction of transactions, account openings, or transfers that are fraudulent within an overwhelming majority of legitimate activity. The defining property of the problem is extreme class imbalance: in the datasets studied in this thesis, fraud accounts for between roughly 0.13% and 1.5% of records (see Section 5.1), and on the European credit-card benchmark only about 0.17% of transactions are fraudulent [1]. Surveys of imbalanced learning have long noted that such skew breaks the assumptions of standard classifiers, which are biased toward the majority class when minimising aggregate error [2].

A first consequence is that plain accuracy is meaningless as an evaluation metric. A trivial classifier that labels every transaction legitimate achieves over 99.8% accuracy on a 0.17%-fraud dataset while detecting nothing. Receiver-operating-characteristic (ROC) analysis is the standard remedy, but the ROC curve and its area can be optimistic under heavy skew because the false-positive axis is normalised by the large negative count [3]. This thesis therefore reports the area under the precision-recall curve (PR-AUC; equivalently, average precision) as its primary metric, since precision is sensitive to the positive base rate and rewards a classifier for ranking the rare fraud cases ahead of the legitimate ones [4], [5].

A second consequence concerns the operating point. In practice an alert is reviewed at a cost, so a fraud detector is deployed at a fixed, low false-positive rate (FPR) rather than at an arbitrary 0.5 threshold. Accordingly, the secondary metrics are recall at 1% FPR and recall at 5% FPR: the fraction of fraud caught while incurring at most that many false alarms among legitimate transactions [6], [7]. These metrics are computed conservatively from the empirical ROC curve, taking the highest achievable recall among thresholds whose FPR does not exceed the budget, rather than interpolating between curve vertices that do not correspond to any realisable threshold (this thesis’s experiments; see Section 4). Classical responses to imbalance — resampling techniques such as SMOTE [8] and loss reweighting such as focal loss [9] — all operate during training and require the model to be retrained for each objective.

1.2 Gradient-boosted trees and the rise of tabular foundation models

For tabular prediction, gradient-boosted decision trees (GBDTs) remain the entrenched baseline. XGBoost [10], CatBoost [11], and LightGBM [12] are routinely competitive with or superior to deep networks on heterogeneous tabular data, a robustness attributed to their handling of uninformative features and irregular target functions [13]. This thesis uses XGBoost and CatBoost, trained on the full training set, as its reference detectors.

A recent and disruptive alternative is the tabular foundation model (FTM). Building on the observation that large transformers [14] perform few-shot learning purely in-context — conditioning on examples supplied at inference time without any weight update [15] — Prior-Data Fitted Networks (PFNs) train a transformer on samples from a prior over datasets so that a single forward pass approximates Bayesian posterior-predictive inference [16], [17]. TabPFN applied this idea to tabular classification [18], [19], and TabPFN v2 was shown to match or beat tuned GBDTs on small and medium tables while requiring no per-dataset training [20]. The family has since scaled

and diversified — TabPFN 2.5 with a larger context window [21], [22], analyses of v2’s behaviour [23], real-data continued pre-training [24], scaling extensions [25], and the distinct TabICLv2 architecture [26]. FTMs already appear in financial applications [27], [28]. This thesis evaluates five FTMs: TabPFN v2, 2.5, 2.6, and v3, plus TabICLv2.

1.3 The problem: a fixed context window wasted under imbalance

In-context learning (ICL) FTMs do not learn parameters from a training set; they read it as a sequence of in-context examples — the *context* — and process it through self-attention at inference. This context window is fixed and modest: roughly 10,000 rows for TabPFN v2 and 48,000–50,000 rows for the larger models in the grid (this thesis’s experiments; see Section 4.4). When the training set exceeds the window, it must be subsampled. The standard strategy is uniform random subsampling.

Under heavy class imbalance this strategy is wasteful. Sampling 10,000 rows at the natural 0.17% fraud rate yields, in expectation, only about 17 fraud examples; the remaining 9,983 slots are spent on legitimate transactions the model has already seen many of. Because the FTM forward pass is a posterior prediction conditioned on this context [16], a context starved of the minority class can only produce a poorly informed posterior for that class. Moreover, what enters the context is itself a controllable inference-time lever — a property recently demonstrated adversarially for FTMs [29]. This thesis exploits the same lever for a benign, imbalance-motivated end. Two distinct deficiencies are at play: the context may be *irrelevant* (random rows are not structurally similar to the transactions being scored) and *imbalanced* (few fraud rows). RAP addresses both.

1.4 The idea: Retrieval-Augmented Prediction

This thesis proposes **Retrieval-Augmented Prediction (RAP)**, a test-time context-construction strategy for ICL FTMs under heavy class imbalance. Rather than subsampling the context randomly, RAP constructs, per test batch, a context that is both structurally relevant and label-balanced. It proceeds in two steps. First, *retrieval*: for each batch of test rows, k-nearest-neighbour (kNN) retrieval in standardised feature space selects training rows similar to the batch, so the context is locally relevant to the transactions being scored. Second, *controlled fraud-ratio oversampling*: the retrieved pool is rebalanced to a target fraud ratio, supplementing scarce neighbourhood fraud from the global training-fraud pool and, when unique fraud is exhausted, duplicating fraud rows with replacement so that oversampling is the method rather than a silent cap (this thesis’s experiments; see Section 4). The first step — relevance retrieval — is by now established for tabular models: TabR couples a deep network with kNN retrieval [30], TabDPT bakes retrieval into a tabular foundation model [31], and LoCalPFN retrieves a local neighbourhood for TabPFN [32]. RAP’s distinctive lever is the second step: deliberately *controlling the fraud ratio* of the retrieved context and sweeping it — an inference-time analogue of SMOTE-style oversampling [8] that, unlike SMOTE, duplicates real transactions rather than synthesising them and requires no retraining of the frozen FTM. No prior retrieval-augmented tabular method tunes the in-context label composition for imbalance.

RAP is decomposed into an ablation that isolates each effect. A random-subsampling baseline (condition C1) is compared against retrieval at the natural fraud ratio (C3) to measure the *relevance*

benefit — C3 is essentially the frozen kNN-context baseline already studied for TabPFN [32] — and C3 against retrieval with ratio control (C4) to measure the *ratio-tuning* benefit, which is the thesis’s novel lever. A stratified 50/50 subsampling condition (C2) isolates the ratio effect without retrieval.

1.5 Contributions

This thesis makes the following contributions.

1. A clear articulation of why uniform random context subsampling is a poor fit for ICL FTMs under heavy class imbalance, framed through the posterior-predictive view of the FTM forward pass.
2. RAP, a test-time context-construction method for frozen ICL FTMs that augments relevance-based kNN retrieval — established for tabular ICL by TabR, TabDPT, and LoCalPFN [30], [31], [32] — with *controlled, swept in-context fraud-ratio oversampling*, the component without precedent in that literature.
3. An empirical evaluation across five FTMs and two GBDT baselines on six public financial-fraud datasets, using prevalence-corrected PR-AUC and conservative recall-at-fixed-FPR metrics.
4. An ablation that decomposes RAP’s benefit into a (largely confirmatory) relevance component and a (novel) ratio component, plus a sweep of the target fraud ratio — directly testing whether two results from adjacent settings hold under heavy tabular imbalance: that random context selection is a strong baseline [33] and that ratio rebalancing of in-context examples is ineffective [34].

1.6 Research questions

The evaluation is organised around four research questions (RQs), mapped to the experimental conditions described in Section 4.

- **RQ1 — FTMs versus GBDTs under random subsampling.** How do ICL FTMs, given a randomly subsampled context (C1), compare to GBDTs trained on the full dataset, measured by PR-AUC and recall at fixed FPR? This establishes the baseline in the specific fraud / prevalence-corrected-PR-AUC regime that prior FTM benchmarks (balanced, accuracy-oriented) do not cover.
- **RQ2 — Does random subsampling degrade at scale?** Does FTM detection under random subsampling (C1) deteriorate as dataset size grows, relative to the unconstrained GBDT baselines? This re-confirms, for the modern FTM line under fraud-scale imbalance, a context-scaling degradation already reported for TabPFN [31], [32].
- **RQ3 — Does retrieval improve over random subsampling?** Does kNN-retrieved context (C3) improve PR-AUC over random subsampling (C1), isolating the relevance effect? This is largely *confirmatory*: a frozen kNN context already beats a random one for TabPFN [32]. RQ3 re-tests that on modern FTMs under heavy imbalance, and against the competing finding that random context selection is a strong baseline that smart selection struggles to beat [33].
- **RQ4 — What is the optimal context fraud ratio?** Across the C4 sweep over target fraud ratios {5%, 10%, 20%, 30%, 50%}, which ratio maximises detection, and does ratio tuning add value beyond retrieval alone (C4 versus C3)? This is the thesis’s **primary, novel** question: no

retrieval-augmented tabular method controls the in-context label ratio, and an analogous result in language-model ICL found ratio rebalancing ineffective [34], so the outcome is genuinely open.

RQ1–RQ3 establish and confirm the setting; RQ4 is the central contribution. The RAP conditions (C3 and C4) are implemented in the next phase of the project, so results for RQ3 and RQ4 are pending.

1.7 Thesis outline

The remainder of the thesis is structured as follows. Section 2 reviews the necessary background: in-context learning, the PFN and TabPFN line of work, the GBDT baselines, and the class-imbalance and evaluation literature. Section 5.1 documents the six fraud-detection benchmarks, their provenance, and their split protocols, including the deliberately retained time features and the one synthetic dataset found to be degenerate. Section 4 specifies the experimental methodology: the four context-construction conditions, the five FTMs and their context limits, the prevalence-corrected metrics, and the RAP pipeline design. The results chapter reports the four research questions in turn. The final chapters discuss limitations — including single-GPU tractability compromises and the questionable meaningfulness of distance-based retrieval over ordinal-encoded entity identifiers — and conclude.

2 Background

This chapter assembles the conceptual foundations the rest of the thesis builds on. It moves from the problem (supervised fraud detection under extreme class imbalance) through the model families involved (gradient-boosted decision trees, then in-context-learning tabular foundation models) to the precise structural limitation that Retrieval-Augmented Prediction (RAP) is designed to address: the fixed context window of an in-context-learning model and how random subsampling of that context starves it of the minority class. Every domain term and acronym is defined on first use.

2.1 Supervised fraud detection under extreme class imbalance

Fraud detection is, in this thesis, a supervised binary classification problem. Each transaction (or account-opening application) is a feature vector $x \in \mathbb{R}^d$ with a label $y \in \{0, 1\}$, where $y = 1$ denotes fraud (the *positive* or *minority* class) and $y = 0$ denotes a legitimate event (the *negative* or *majority* class). A model produces a score $s(x) \in [0, 1]$ interpreted as $\Pr(y = 1 \mid x)$, and a decision is taken by thresholding that score. The defining property of the setting is *extreme class imbalance*: the prevalence $\pi = \Pr(y = 1)$ is very small. Across the six benchmark datasets used in this thesis it ranges from roughly 0.13% to 1.5%, and in the most extreme case (the PaySim mobile-money dataset) it is about 0.13%; the European credit-card benchmark, at about 0.17%, has only around 98 fraudulent transactions in a held-out test set [1]. Such skew breaks the naive baseline: a classifier that predicts “legitimate” for every input attains over 99% accuracy while detecting no fraud at all. Accuracy is therefore uninformative here, and the imbalanced-learning literature has long argued that both the learning algorithm and its evaluation must be adapted to the minority class [2].

2.1.1 Evaluation metrics

Because accuracy is degenerate under imbalance, this thesis evaluates models with threshold-free, rank-based metrics. The receiver operating characteristic (ROC) curve plots the true-positive rate (TPR, equivalently recall) against the false-positive rate (FPR) as the decision threshold is swept, and the area under it (ROC-AUC) summarises ranking quality independent of any single threshold [3]. ROC-AUC has a known weakness under heavy imbalance, however: because the FPR is normalised by the large pool of negatives, a model can move many false positives in absolute terms while barely perturbing the FPR, so ROC-AUC can look optimistic when the positive class is tiny [3]. The precision-recall (PR) curve instead plots precision against recall; precision depends directly on the absolute number of false positives relative to true positives, so the PR curve and the area under it (PR-AUC, equivalently average precision) are far more sensitive to minority-class performance and are the preferred summary under severe skew [4], [5].

This thesis reports **PR-AUC** (average precision) as its primary metric, supplemented by two operating-point metrics, **Recall@1%FPR** and **Recall@5%FPR**, and by ROC-AUC and F1 as secondary diagnostics. The recall-at-fixed-FPR metrics reflect the operational reality of fraud screening, where an institution can only manually review a small, fixed fraction of flagged transactions and therefore fixes a low false-positive budget [6]. In this thesis’s experiments these metrics use a deliberately conservative definition: Recall@ $k\%$ FPR is the highest TPR among the operating points of the *empirical* ROC curve whose FPR is at most the target. Linear interpolation

between empirical ROC vertices is avoided on purpose, because an interpolated (FPR, TPR) pair corresponds to no achievable decision threshold and thus overestimates attainable recall – an effect that is pronounced when the positive class is small and the ROC is coarse, as in the European credit-card dataset. The methods chapter returns to this and to the prevalence correction applied to PR-AUC under test-set subsampling.

2.1.2 Classic imbalance remedies

The imbalanced-learning literature offers three broad families of remedy [2]. *Data-level* methods rebalance the training distribution; the canonical example is SMOTE (Synthetic Minority Over-sampling Technique), which generates new minority examples by interpolating between a minority point and its nearest minority neighbours rather than merely duplicating rows [8]. *Algorithm-level* or *cost-sensitive* methods leave the data alone but reweight the learning objective so that errors on the minority class are penalised more heavily; focal loss is a widely used instance that down-weights easy, well-classified majority examples and concentrates training on hard minority examples [9]. *Ensemble* and clustering approaches combine resampling with multiple learners. A recurring finding on these benchmarks is that, under extreme imbalance, the choice of training objective can matter more than the choice of architecture [35].

These remedies share an assumption that the model is being *trained*: SMOTE alters the training set, focal loss alters the loss minimised during training, and cost weights enter gradient updates. That assumption does not hold for the in-context-learning models at the centre of this thesis, whose weights are frozen at inference time. RAP can be read as a data-level remedy relocated to inference: rather than resampling a training set before fitting, it constructs a relevant, label-balanced *context* at prediction time. The next sections build up to why such a model exists and why its context is the natural place to intervene.

2.2 Tabular machine learning and gradient-boosted decision trees

Most fraud data is *tabular*: heterogeneous columns of numeric and categorical features, no spatial or sequential structure that convolutional or recurrent inductive biases would exploit. For two decades the dominant approach to tabular prediction has been the gradient-boosted decision tree (GBDT), an additive ensemble that fits each new shallow tree to the gradient of the loss left by the current ensemble. XGBoost provides a scalable, heavily regularised implementation that became a default strong baseline across tabular competitions and applications [10]. CatBoost contributes ordered boosting (a permutation-driven scheme that mitigates target leakage) and native handling of categorical features, which is useful given that several fraud datasets carry categorical columns such as transaction type or device operating system [11]. LightGBM rounds out the family with histogram-based splitting tuned for speed on large datasets [12].

Deep learning was slow to displace GBDTs on tabular data. A careful empirical study attributes the gap to properties of tabular targets: they are often irregular, non-smooth functions of the inputs, they contain uninformative features that neural networks struggle to ignore, and they lack the locality or invariance structure that gives deep models their edge on images and text [13]. GBDTs, by contrast, are rotationally non-invariant axis-aligned learners that handle uninformative features and irregular boundaries gracefully. This robustness is exactly why this thesis treats

XGBoost and CatBoost as strong baselines rather than as straw men. They are trained on the full training set with no context limit, because the limited context window is an inherent property of the foundation models being evaluated, not a constraint that should be imposed on the baselines. The central empirical question — whether context-aware in-context learning can close the gap to GBDTs under heavy imbalance — only has meaning against such well-tuned tree baselines.

2.3 In-context learning and Prior-Data Fitted Networks

The foundation-model side of the thesis descends from the Transformer, an architecture that replaces recurrence with self-attention so that every element of an input set can attend to every other element in a single forward pass [14]. Self-attention is the substrate on which the tabular foundation models studied here operate: a row’s prediction is computed by attending over a set of labelled rows supplied alongside it. A pivotal observation about large Transformer language models was that they perform *few-shot learning purely in-context* — given a handful of input-output examples in the prompt at inference time, they generalise to new inputs without any gradient update to their weights [15]. This is *in-context learning* (ICL): the examples conditioned upon, not the weights, carry the task-specific information.

Prior-Data Fitted Networks (PFNs) give ICL a Bayesian interpretation [16]. A PFN is a Transformer trained on synthetic datasets drawn from a prior over data-generating mechanisms; its training objective makes it approximate the *posterior predictive distribution* — the probability of a query label given the conditioned examples, marginalised over the prior — in one forward pass. The expensive Bayesian inference is amortised into pre-training; at inference the network simply conditions on the supplied examples and reads off an approximate posterior. The open PFN training infrastructure that underlies this line of work is publicly maintained [17].

TabPFN applied PFNs to tabular classification [18], [19]. A single pre-trained Transformer takes a small labelled training table as in-context examples and classifies a query table in one forward pass, with no per-dataset training and no hyperparameter search. The original model targeted the small-data regime and carried a modest cap on the number of in-context rows it could ingest — the very constraint that, scaled up to heavily imbalanced data, RAP works within. The Bayesian framing is the lens through which the rest of this thesis reads the context: if the forward pass approximates a posterior predictive conditioned on the supplied examples, then *which* examples are supplied directly shapes the posterior, and a context that is both relevant to the query and adequately populated with the minority class should yield a sharper posterior under imbalance.

2.4 Tabular foundation models

The TabPFN line has since matured into a family of *tabular foundation models* (FTMs) — pre-trained models intended to be applied off-the-shelf across many tabular tasks. TabPFN v2, published in *Nature*, scaled the prior, the architecture, and the supported data size, and reported matching or exceeding tuned GBDT baselines on small and medium tabular datasets while running in seconds and requiring no per-dataset tuning [20]. Subsequent releases pushed capacity further: TabPFN 2.5 enlarged both the context capacity and accuracy over v2 [21], [22], and TabPFN 2.6 and v3 continued the series within the same model family [36], [37]. A parallel architecture, TabICLv2, offers an alternative open ICL tabular model with a large context capacity,

providing a useful test of whether context-construction effects generalise beyond the TabPFN family [26]. Active research extends FTMs along orthogonal axes — continued pre-training on real rather than purely synthetic data [24], scalability and ensembling strategies [25], resilience to distribution drift [38], analyses of why TabPFN v2 works and where it fails [23], and training-free relational extensions [39]. RAP is complementary to all of these: it improves the *test-time context* rather than the model weights or the prior.

Mechanically, an ICL FTM does classification by *conditioning on a training context at inference*. The model receives a context $(X_{\text{ctx}}, y_{\text{ctx}})$ of labelled rows together with the unlabelled query rows; a single forward pass attends across the context and emits class probabilities for the queries. There is no fitting in the GBDT sense — what the wrappers in this thesis call `fit` merely loads the context into the frozen model, and `predict_proba` performs the conditioned forward pass. Because the context is supplied at inference, it is a fully controllable lever: prior work has shown, in an adversarial setting, that perturbing the in-context set can steer FTM predictions, demonstrating that the context is a genuine attack surface and, equivalently, a genuine design surface [29]. FTMs are already entering tabular benchmarking leaderboards alongside the classical baselines [40], are surveyed within the broader arc of representation learning for tabular data [41], and are appearing in applied financial settings, from Ethereum fraud detection [27] to an industrial financial-risk deployment [28]. This thesis asks whether they can be made competitive fraud detectors specifically under heavy imbalance, and whether the lever for doing so is the context.

2.5 The context-window bottleneck under imbalance

The crux of the thesis is a structural mismatch between how an ICL FTM consumes a context and what extreme imbalance does to a randomly drawn one. Each FTM supports only a bounded number of context rows. The models evaluated here, with their soft context limits (the upper end of their pre-training range, in rows), are summarised below; these limits are read from the model wrappers used in the experiments.

Model	Context limit (rows)	Notes
TabPFN v2	10,000	smallest context in the grid
TabPFN 2.5	50,000	enlarged context vs v2
TabPFN 2.6	50,000	same family capacity
TabPFN v3	50,000	provisional, pending checkpoint confirmation
TabICLv2	48,000	no hard cap; pre-training upper bound

Source: *model wrappers in this thesis’s codebase* (`src/models/ftm.py`); *context limits per* [20], [21], [26].

When the training set is larger than the context limit, the baseline strategy (condition C1 in this thesis) is to draw a uniform random subsample of the training rows up to the limit. Under heavy imbalance this subsample inherits the population prevalence in expectation, which is the problem. If the prevalence is π and the context holds c rows, the expected number of fraud rows in a

uniform random context is πc . For TabPFN v2’s 10,000-row context, a 0.13%-prevalence dataset such as PaySim [42] yields about 13 expected fraud rows, and a 0.17%-prevalence dataset such as the European credit-card benchmark [1] yields about 17; at the larger 48,000–50,000-row contexts those counts rise only to roughly 62–65 and 82–85 respectively. The context window is therefore flooded with legitimate transactions while the minority class — the only class the task actually cares about — is represented by a handful of rows. The exact counts depend on dataset prevalence and the per-model limit; representative values are tabulated below.

Dataset (prevalence)	Expected fraud rows @ 10k	@ 48k	@ 50k
PaySim (0.13%)	~13	~62	~65
eu_cc (0.17%)	~17	~82	~85
BAF (1.1%)	~110	~528	~550
BankSim (1.2%)	~120	~576	~600

Expected fraud count = πc for a uniform random context of size c ; prevalences per the dataset chapter of this thesis.

This is the precise gap RAP targets. Two distinct deficiencies are at work, and the experiment design (detailed in the methods chapter) is structured to separate them. First, a uniform random context is *irrelevant* to a given query: it is a random slice of the whole training distribution rather than the region of feature space the query inhabits. RAP addresses this with k-nearest-neighbour (kNN) retrieval, building a context from the training rows nearest to each test batch. Second, even a relevant context inherits the population prevalence and so remains starved of fraud; RAP additionally rebalances the retrieved context to a controlled fraud ratio by oversampling minority rows. The first intervention echoes the few-shot insight that the supplied examples carry the task signal [15], applied at the granularity of the individual query; the second is a data-level imbalance remedy [2] relocated from training time to inference time, conceptually adjacent to SMOTE-style oversampling [8] but operating on the FTM’s context rather than on a training set. Because the FTM forward pass approximates a posterior predictive conditioned on its context [16], improving the relevance and class balance of that context is a principled way to sharpen the posterior under extreme imbalance — without retraining, reweighting, or altering the frozen model in any way.

The methodological building blocks RAP relies on are themselves standard. kNN retrieval over feature space, demonstration selection for in-context learning, and the choice of distance metric and feature scaling for that retrieval are established techniques [43], [44], [45], [46] whose specific instantiation in this thesis is deferred to the methods chapter. What matters for the present chapter is the framing: a frozen ICL tabular foundation model, evaluated under prevalence around 0.1–1%, with a fixed context window that random subsampling fills almost entirely with the majority class. RAP is the test-time strategy that fills that window differently.

3 Related Work

This chapter positions Retrieval-Augmented Prediction (RAP) against four threads of prior work. The first traces the lineage of the TabPFN family of tabular foundation models (FTMs) and the parallel effort to scale their in-context-learning (ICL) capacity. The second surveys FTMs and transformers applied to fraud and finance. The third reviews classical and deep fraud detection together with the broader class-imbalance literature, including the anti-money-laundering (AML), graph, and synthetic-data work that surrounds it. The fourth frames RAP itself as retrieval-augmented ICL specialized to heavily imbalanced tabular data. Each thread closes with the gap it leaves, and those gaps jointly motivate the contribution of this thesis.

3.1 The TabPFN lineage and the scaling of context

The line of work this thesis builds on begins with the observation that transformers [14] perform few-shot learning purely in context, conditioning on examples supplied at inference without any weight update [15]. Müller et al. [16] recast this as approximate Bayesian inference: a Prior-Data Fitted Network (PFN), trained on samples drawn from a prior over datasets, approximates the posterior predictive distribution in a single forward pass. TabPFN ported the PFN recipe to tabular classification [18], [19], and TabPFN v2 demonstrated that a single pre-trained transformer matches or surpasses tuned gradient-boosted decision trees (GBDTs) on small and medium tables [20]. The PFN training methodology and reference implementations are maintained in open repositories [17], [47].

The family has since advanced along two axes. The first is the model itself: TabPFN-2.5 enlarges the usable context and improves accuracy over v2 [21], [22], analyses of v2 dissect why it works and where it fails [23], Real-TabPFN improves the prior by continued pre-training on real-world tables rather than purely synthetic ones [24], and Drift-Resilient TabPFN extends ICL to temporal distribution shift [38]. TabICLv2 offers an alternative ICL architecture with a large, open context and faster inference [26]. A complementary direction widens the data substrate beyond single tables: a training-free relational-database foundation model argues that strong relational predictions need no bespoke pre-training [39]. The benchmarking and survey backdrop — living tabular leaderboards [40] and a survey of representation learning for tabular data [41] — frames where these models currently stand relative to the GBDT baselines [10], [11], [12], [13] this thesis also evaluates.

The second axis is scale, which is the one most relevant here. TabPFN Unleashed proposes scalability and effectiveness extensions, including context-handling and ensembling strategies, to push TabPFN toward larger problems [25]. The official extension tooling [48] and the model-version progression measured in this thesis — TabPFN v2 at roughly a ten-thousand-row context window and TabPFN 2.5, 2.6, v3, and TabICLv2 at roughly forty-eight to fifty thousand rows [37], [40] — both treat the context as something to make *larger*. None of this work, however, treats the context as something to *compose*. The implicit assumption is that, given more capacity, a representative subsample of the training set suffices. Under heavy class imbalance that assumption breaks: a random subsample drawn at the natural fraud rate of roughly one in a thousand floods even a fifty-thousand-row window with legitimate transactions and may contain only a handful of fraud rows, or none. The gap this thread leaves is precise: no member of the TabPFN lineage addresses *test-*

time context construction under extreme imbalance, where which rows enter the window matters more than how many it can hold.

3.2 Foundation models and transformers in fraud and finance

A small but growing body of work applies FTMs and related transformers to financial-crime tasks, establishing both feasibility and the practical stakes of improving them. Olusegun et al. apply an explainable TabPFN-style tabular transformer to Ethereum fraud detection and report improved detection with interpretability, demonstrating that a frozen FTM is a viable fraud classifier [27]. On the industrial side, a Prior Labs case study describes deploying TabPFN for financial risk management in collaboration with Taktile [28], and the vendor documents an unsupervised anomaly-detection mode for the same model [49]. Beyond single-model classification, a multi-agent debate framework has been proposed for tabular anomaly detection [50], signaling interest in composing FTMs into larger detection systems.

Two findings in this thread bear directly on the present design. First, the robustness study of Djilani et al. shows that the in-context set is a controllable test-time lever: adversarial manipulation of the supplied context degrades FTM predictions, and the proposed defenses also operate by editing the context [29]. This establishes, academically, that *what enters the context changes the prediction* — RAP is a benign, imbalance-motivated instance of the same mechanism. Second, the anomaly-detection framing [49] is unsupervised, whereas fraud detection here is supervised with scarce but informative labels. The gap this thread leaves is that existing FTM-for-fraud work either takes the context as given (single-model classification and case studies) or abandons supervision (anomaly detection); none deliberately *constructs* a supervised, label-balanced, relevance-targeted context to counteract imbalance, even though the robustness literature shows the context is exactly the right surface to act on.

3.3 Classical and deep fraud detection and class imbalance

Fraud-detection research has largely pursued an architectural and objective-level arms race. Recent deep models include energy-based Restricted Boltzmann Machines combined with xLSTM for real-time card fraud [51], DBSCAN-augmented disjunctive voting ensembles that use density-based clustering to separate the minority class [52], and deep networks paired with focal loss and resampling on the European card benchmark [53]. Relational and structured systems take a different route: a three-level relationship-penetration support system uncovers coordinated fraud through entity structure [54], and graph-database integration has been demonstrated on the BankSim payment simulator [55]. A closely related and instructive finding comes from Sun et al., who argue that under extreme imbalance on the Bank Account Fraud (BAF) suite the training objective matters more than the model architecture [35]. That conclusion motivates the stance taken here: rather than add another architecture, RAP intervenes on what the model conditions on at inference.

A parallel strand addresses financial crime at the network level. Realistic synthetic transaction graphs have been generated for AML model development with controllable laundering typologies [56]; graph-neural methods detect laundering on transaction graphs, including a multi-aggregation Graph Isomorphism Network [57] and temporal graph networks for time-evolving financial

anomalies [58]; knowledge-based multi-agent frameworks both generate and detect laundering patterns [59]; and federated learning has been used to detect mule accounts across institutions without sharing raw data [60]. Applied machine-learning studies on individual benchmarks, such as PaySim [61], round out the conventional-baseline picture. This thesis deliberately operates in the purely tabular, single-table feature space and treats these graph, relational, and federated approaches as complementary rather than competing.

The imbalance methodology underpinning all of this is well established. Classical resampling synthesizes minority examples by interpolating between near neighbours (SMOTE) [8]; objective-level methods reweight the loss to focus on hard minority examples (focal loss) [9]; and surveys catalogue the sampling, cost-sensitive, and algorithm-level families together with appropriate evaluation metrics [2]. The evaluation protocol of this thesis follows the ROC and precision-recall tradition for skewed data, which warns against accuracy and naive thresholds [3] and motivates the use of PR-AUC (area under the precision-recall curve), Recall at fixed false-positive rate (FPR), and ROC-AUC [4], [5], [6]. The gap this thread leaves is methodological placement. Resampling and reweighting both presuppose a model that is *trained* on the rebalanced data or objective; an ICL FTM is frozen and cannot be retrained per objective. RAP can be read as relocating SMOTE-style oversampling and cost-sensitive intent from training time into test-time context construction, but it differs in that the oversampling duplicates real fraud rows with replacement rather than synthesizing interpolated ones, and it couples that rebalancing with relevance-based retrieval — a combination the classical imbalance literature does not address for frozen ICL models.

3.4 Retrieval-augmented and context-selection for in-context learning

The fourth thread is the one RAP most directly extends. RAP retrieves a structurally similar context for each test batch using k-nearest-neighbour (kNN) retrieval in scaled feature space, then enforces a controlled fraud ratio by oversampling. The retrieval half is, by now, well established for tabular models. TabR augments a tabular deep network with a kNN-like retrieval module and matches or beats gradient-boosted trees [30]; TabDPT trains retrieval directly into a tabular foundation model, retrieving a query-specific context at both training and inference time [31]; and LoCalPFN retrieves a local kNN neighbourhood for TabPFN and then fine-tunes the model on it [32]. Crucially, LoCalPFN’s frozen, retrieval-only variant (its “TabPFN-kNN” ablation) is *exactly* a frozen kNN context for TabPFN, and it already shows retrieval beating a random context — so neither retrieval-augmented tabular ICL nor its frozen, test-time form is novel to this thesis. The closest work on the imbalance axis, AWARE, *learns* a task-aligned retrieval embedding for tabular ICL and reports that naive distance-based retrieval degrades under severe class imbalance because neighbourhoods skew to the majority class [62]; but it trains that embedding per dataset, targets clinical electronic-health records rather than fraud, and never controls the in-context class ratio. What is absent from all of this work is deliberate control of the *label composition* of the retrieved context — the lever RAP adds. Within the broader FTM literature, the robustness study cited above is the nearest precedent for treating the in-context set as a controllable input [29], and TabPFN Unleashed touches retrieval-style context handling as a scaling device [25], but neither targets imbalance.

On the language-model side, the sensitivity of ICL to *which* demonstrations are supplied is well documented: retrieving semantically similar examples by kNN improves few-shot performance over random selection [43], a learned retriever improves it further [44], and even the *order* of the demonstrations can swing accuracy from near state-of-the-art to near chance [63], [64]. These results establish context composition as a first-class lever — the premise RAP builds on. The retrieval machinery itself is classical: nearest-neighbour classification dates to T. M. Cover and P. E. Hart [45], and standardising features before computing distances, so that no single high-variance feature dominates the metric, is standard practice [46].

Two findings from adjacent settings make RAP’s central question genuinely open rather than a foregone conclusion. On balanced tabular data, random sketching of the TabPFN context is a strong baseline that smart *global* selection (k-means, coresets) fails to beat [33]; and in language-model ICL, rebalancing the class ratio of the demonstration pool — the very lever RAP pulls — was found *ineffective*, with stratified 50/50 selection actually reducing accuracy [34]. Whether per-batch retrieval and explicit ratio control overturn these results under heavy *tabular* imbalance is precisely what RAP tests; a null result would itself extend these findings to a new regime.

Two features distinguish RAP within this thread, and only the second is novel. First, retrieval — confirmatory: condition C3 reproduces the frozen kNN context already studied for TabPFN [32] and re-tests it on modern FTMs under heavy imbalance. Second, and the genuine contribution, retrieval is coupled with explicit fraud-ratio control: condition C4 sweeps target ratios (5, 10, 20, 30, and 50 percent) and fills the fraud quota with replacement when unique training fraud is exhausted, so oversampling is the method rather than a fallback. The only prior method to rebalance a retrieved ICL context for imbalance, IM-Context, does so for *regression* with a fixed inverse-density mixture and no ratio sweep, on small early ICL backbones [65]; RAP instead treats the in-context fraud ratio as a tunable quantity for binary classification on modern tabular FTMs. The design is also decomposable: C3 versus C1 isolates the relevance effect, C4 versus C3 the ratio effect, and C2 (stratified, no retrieval) gives a retrieval-free isolation of the ratio effect — an attribution prior context-scaling work does not attempt. The retrieval and ratio-control conditions (C3 and C4) are not yet implemented at the time of writing; the corresponding results are pending and reported in later chapters as part of this thesis’s experiments.

3.5 Summary of gaps

Across the four threads, the niche is precise. Retrieval-augmented tabular ICL is established — including its frozen, test-time form (LoCalPFN’s TabPFN-kNN) [31], [32] — and a learned-retrieval method has begun to address imbalance, but in the clinical domain and via per-dataset training [62]. What no work does is treat the in-context *label ratio* as a controlled, swept lever for frozen off-the-shelf tabular FTMs under heavy class imbalance; and the two most relevant adjacent results point in opposite directions — random selection is hard to beat [33], yet ICL ratio-rebalancing can fail [34]. Resolving whether controlled fraud-ratio context construction helps frozen tabular FTMs on fraud — and decomposing any gain into a relevance and a ratio component — is the contribution of this thesis. The thesis is therefore deliberately positioned so that its novelty rests on the ratio-control sweep (RQ4) and the relevance/ratio decomposition, with the relevance effect (RQ3) framed as a confirmatory re-test rather than a discovery.

4 Methods

This chapter specifies the experimental design for evaluating Retrieval-Augmented Prediction (RAP). Section 4.1 formalises test-time context construction for an in-context-learning (ICL) tabular foundation model (FTM) under a context budget and extreme class imbalance. Section 4.2 describes the RAP pipeline (retriever, sampler, context builder, batching). Section 4.3 defines the four experimental conditions and the decomposition that separates retrieval relevance from fraud-ratio control. Section 4.4 lists the models and the rationale for the gradient-boosted-decision-tree (GBDT) baseline protocol. Section 4.5 defines the evaluation metrics. Throughout, the implemented baseline conditions (C1, C2) are distinguished from the RAP conditions (C3, C4), whose design is fixed here but whose results are pending the Phase 3 implementation.

4.1 Problem formulation

An ICL FTM such as TabPFN [20] or TabICLv2 [26] makes a prediction in a single forward pass, conditioning on a set of labelled examples supplied at inference time rather than updating its weights. This is the few-shot, in-context paradigm established for large language models [15] and given a Bayesian interpretation by Prior-Data Fitted Networks (PFNs), which approximate the posterior predictive distribution conditioned on the supplied examples [16]. We make this conditioning explicit. Let a labelled training pool be $D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$ with binary labels $y_i \in \{0, 1\}$ (1 = fraud). For a test point x , the FTM computes $f(x \mid C)$, where the *context* $C = (X_{\text{ctx}}, y_{\text{ctx}})$ is a subset of (or a resampling from) D_{train} . Because attention cost grows with the number of in-context rows [14], every FTM imposes a **context budget** B : a soft upper bound on the number of context rows it was pre-trained to handle (Section 4.4). When $n > B$, the practitioner must choose which rows enter C . That choice is the object of study.

Two properties of the fraud-detection setting make the choice consequential. First, the **prevalence** $\pi = \Pr[y = 1]$ is extreme: across the formal-grid datasets it ranges from 0.13% to 1.5% (Section 4.4). Second, $n \gg B$ on every formal dataset, so a context that respects the budget contains only a small slice of the training pool. Under the naive strategy of drawing B rows uniformly at random, the expected number of fraud rows in the context is $B\pi$. For TabPFN v2 ($B = 10,000$) on PaySim ($\pi \approx 0.0013$) this is roughly 13 fraud examples; the remaining 9,987 rows are legitimate transactions, most of them uninformative for the decision boundary. The context window is, in effect, flooded with the majority class. RAP is a response to this: a test-time strategy that builds C to be both *relevant* (structurally similar to the test point) and *label-balanced* (containing a controlled fraction of fraud), without retraining the FTM.

This places RAP within the established taxonomy of imbalance handling [2], which spans data-level resampling [8], cost-sensitive losses [9], and algorithm-level changes. RAP is a data-level intervention, but applied at *inference* to the in-context set rather than at *training* to a model’s data — a distinction that matters for frozen FTMs, which cannot be re-fitted per objective. That the in-context set is a controllable test-time lever has been demonstrated adversarially for tabular FTMs [29]; RAP is a benign, imbalance-motivated instance of the same mechanism. It is also complementary to model-side advances such as continued pre-training on real data [24] and general

scaling of TabPFN’s context handling [25], because it changes only what enters the context, not the model.

4.2 The RAP pipeline

RAP constructs the context per test batch from three components: a retriever, a sampler, and a context builder. Algorithm 1 gives the full per-batch procedure (from the experiment plan, §5).

```

Inputs: X_train, y_train      (full training pool, unscaled)
        X_test               (test set to be scored)
        B                    (FTM context budget)
        k                    (retrieval pool size)
        r                    (target fraud ratio; None for C3)
        metric                (cosine or euclidean)

Setup (once per split):
    scaler <- StandardScaler().fit(X_train)      # fit on TRAIN only
    Xs_train <- scaler.transform(X_train)
    index <- NearestNeighbors(metric).fit(Xs_train) # built ONCE
    F_global <- {i : y_train[i] == 1}           # global fraud pool

Per-batch inference:
    batches <- cluster_then_chunk(scaler.transform(X_test), B) # homogeneous
    for batch in batches:
        c <- mean(scaler.transform(batch))      # batch centroid (scaled)
        cand <- index.query(c, k)                # k nearest TRAIN indices
        if r is None:                             # C3: keep natural ratio
            idx <- cand
            guard_against_single_class(idx, y_train, F_global)
        else:                                       # C4: enforce ratio r
            n_fraud <- round(r * B)
            fraud <- fraud rows in cand
            if |fraud| < n_fraud:                   # supplement / oversample
                fraud <- fraud + sample(F_global, n_fraud - |fraud|,
                                         with_replacement = (n_fraud > |F_global|))
            legit <- sample(legit rows in cand, B - n_fraud)
            idx <- shuffle(fraud[:n_fraud] + legit)
        X_ctx, y_ctx <- X_train[idx], y_train[idx] # UNSCALED rows
        model.fit(X_ctx, y_ctx)
        probs[batch] <- model.predict_proba(batch)[: , 1]

```

Retriever. Retrieval operates in a `StandardScaler` feature space fit on the training pool only (no test leakage), so that Euclidean and cosine distances are not dominated by high-variance columns. A `NearestNeighbors` index is built once per dataset–seed split and reused across every batch and condition; this amortises the dominant index-construction cost. Each test batch is reduced to its centroid in scaled space, and the top- k nearest training rows are retrieved. The retrieval metric (cosine versus Euclidean) is a configurable factor to be resolved on the development dataset in

Phase 3. A standing caveat applies to BankSim: its `customer` and `merchant` features are ordinal-encoded entity identifiers, and distances over an arbitrary integer coding of identities carry no geometric meaning, so retrieval relevance is expected to be weak for that dataset and will be flagged in the results.

Scaling is for the index only. The scaled features serve solely to define neighbourhoods; the context rows passed to the FTM are the original **unscaled** values. FTMs ingest raw features under all conditions, matching the C1/C2 baselines and avoiding a confound between scaling and retrieval.

Sampler. Under C4, the sampler enforces a target fraud ratio r in the context. It first takes the fraud rows present in the retrieved candidate pool; if these are insufficient to fill the quota $\text{round}(rB)$, it supplements from the global training fraud pool, and when the unique global fraud is itself exhausted it draws **with replacement**, duplicating fraud rows. This with-replacement policy is deliberate: oversampling is the method, and capping the quota at the available unique fraud would silently record a target ratio that the context does not actually realise. The policy is applied uniformly across all datasets; duplication-heavy cells are annotated or excluded at analysis time using a stated duplication threshold, not removed from the grid. To make duplication auditable, every run logs both `context_fraud_n` (the number of fraud *slots* in the context) and `context_fraud_unique` (the number of *distinct* fraud rows); their ratio is the per-run duplication factor. This is analogous in spirit to classical minority oversampling [8], but RAP duplicates existing fraud rows rather than synthesising interpolated ones, keeping every context row a genuine transaction.

Context builder and batching. The builder dispatches on the condition and returns $(X_{\text{ctx}}, y_{\text{ctx}})$ of length $\min(n, B)$. For the retrieval conditions, the test set is partitioned into homogeneous batches before scoring: test rows are clustered in scaled space (e.g. mini-batch k-means, or sorting by the first principal component) and grouped, so that each batch centroid is representative of its members and the retrieved context is genuinely relevant to them. For the non-retrieval baselines (C1, C2) the context is fixed, so test rows are independent given C and batching is purely a memory device with no effect on results.

4.3 Experimental conditions

Four context-construction conditions are evaluated. C1 and C2 are implemented; C3 and C4 are specified here and implemented in Phase 3 (`src/rap/` currently exposes only its package init, and the runner raises `NotImplementedError` for these conditions).

C1 — Random (baseline). If $n \leq B$ the whole training pool is used; otherwise B rows are drawn uniformly at random *without replacement*. The natural fraud ratio is preserved. The context is built once and reused for all test rows.

C2 — Stratified 50:50. The context targets $\lfloor B/2 \rfloor$ fraud rows and the remainder legitimate, sampled without replacement. When unique training fraud is scarcer than the target, the fallback takes *all* fraud rows and fills the rest with legitimate ones; C2 is therefore “as close to 50:50 as the available fraud allows,” not a guaranteed even split. C2 isolates the effect of the context fraud *ratio* relative to C1, holding the (random) relevance fixed.

C3 — kNN, natural ratio (control). Retrieval as in Algorithm 1 with r unset: the context keeps whatever fraud ratio the retrieved neighbourhood happens to contain. A guard is required because a fraud-free retrieved context yields a single-class y_{ctx} , which breaks `predict_proba(...)[:, 1]`; the guard supplements at least one fraud (and one legitimate) row so both classes are present.

C4 — kNN + ratio (full RAP). Retrieval as in C3, followed by the sampler enforcing a target fraud ratio swept over $\{0.05, 0.10, 0.20, 0.30, 0.50\}$.

Decomposition. The conditions form an additive ladder $C1 \rightarrow C3 \rightarrow C4$ that separates two effects. C3 versus C1 isolates the **relevance** effect — the benefit of retrieving a structurally similar context, holding the fraud ratio at its natural value. C4 versus C3 isolates the **ratio** effect — the additional benefit of fraud-ratio tuning, holding retrieval fixed. C2 versus C1 provides a complementary, retrieval-free isolation of the ratio effect. This maps onto the research questions: RQ1 compares FTMs to GBDTs under random subsampling (C1); RQ2 asks whether random subsampling degrades as datasets grow (C1 across size tiers); RQ3 asks whether retrieval improves over random subsampling (C3 versus C1); RQ4 seeks the optimal context fraud ratio (the C4 sweep). Because C3 coincides with the frozen kNN-context baseline already demonstrated for TabPFN [31], [32], RQ3 is largely confirmatory; the C4 ratio sweep (RQ4) is the design’s primary, novel element. GBDT baselines run under C1 only (see Section 4.4).

4.4 Models

Tabular foundation models. Five FTMs are targeted, all run on GPU (`device="cuda"`). Four are members of the TabPFN family [20], [21] and one is the alternative TabICLv2 architecture [26], included to test whether RAP’s benefits generalise beyond TabPFN. Table 1 lists their context budgets, taken from the model wrappers (`CONTEXT_LIMITS`). The runner uses 100% of each budget: `context_size = min(n_train, B)`. Budgets are soft (the pre-training range), not hard caps; the larger 50k-row budgets directly affect C4 feasibility, since at those sizes no formal dataset reaches a fraud ratio of 20% or more without with-replacement oversampling, and the smallest dataset, EU-CC, reaches no swept ratio on any model without it.

Table 1: Tabular foundation models and their context budgets. TabPFN versions are loaded through `tabpfm` 8.0.8; TabICLv2 through `tabicl`. Library versions are recorded per run for provenance.

Model key	Model	Context budget (rows × features)	License
<code>tabpfm_v2</code>	TabPFN v2	10,000 × 500	ungated
<code>tabpfm_25</code>	TabPFN 2.5	50,000 × 2,000	gated (accepted)
<code>tabpfm_26</code>	TabPFN 2.6	50,000 × 2,000	gated (accepted)
<code>tabpfm_3</code>	TabPFN v3	50,000 × 2,000 (provi- sional)	gated (acceptance pending)
<code>tabicl_v2</code>	TabICLv2	48,000 × 500	BSD-3

TabPFN versions are resolved through `TabPFNClassifier.create_default_for_version(...)`, with weights lazy-loaded on first fit and reused thereafter so that one-time setup cost is timed separately from context fitting. TabPFN v3 requires a one-time license acceptance for the token-holding account; until that is granted it is excluded from the grid driver and has no results, so the in-flight grid is effectively four FTMs. Its 50k-row context is the operational budget configured in this thesis’s wrapper, comfortably within TabPFN v3’s architectural capacity — the v3 technical report scales in-context learning to roughly one million training rows [37].

Each FTM prediction averages over `n_estimators` ensemble passes, a deep-ensemble-style averaging that improves stability [66]. The library default is 8, but **all reported FTM grid results use `n_estimators = 4`**, a settled decision for tractability on a single GPU (the grid driver sets it for every FTM run). Reducing the count speeds prediction roughly linearly but changes outputs, and the count is part of the configuration hash; consequently, single ad-hoc runs that omit the flag use the default of 8 and are *not* comparable to grid results — the `n_estimators` column must be checked before any comparison.

GBDT baselines. Two gradient-boosted-decision-tree models serve as strong reference baselines, reflecting the long-standing dominance of tree ensembles on tabular data [13]: XGBoost [10] and CatBoost [11], the latter chosen partly for its native categorical handling given the categorical features in several fraud datasets. Both use 2,000 boosting rounds at depth 6, learning rate 0.05, and subsample 0.8; both auto-set `scale_pos_weight = n_neg / max(n_pos, 1)` to counter imbalance, hold out a stratified 10% of training data for early stopping with patience 50, and use a precision-recall objective for the stopping metric (`aucpr / PRAUC`).

Why GBDTs run under C1 only, on the full training set. The GBDTs are trained on the *entire* training pool with no context limit, and only under C1. This is deliberate, not an oversight: the limited context window is an *inherent property of the FTM architecture* that the thesis sets out to evaluate, not a constraint to impose on every model. The FTM-versus-GBDT comparison therefore measures how an FTM operating under its architectural budget fares against a model that sees all available data — the central tension of whether context-aware ICL can close the gap to GBDTs under heavy imbalance. Applying a context budget to the GBDTs would handicap them artificially and obscure that comparison.

4.5 Evaluation metrics

All metrics are computed from the predicted positive-class probabilities of the scored test set. Under heavy imbalance, accuracy and threshold-dependent measures are misleading, so the protocol follows established ROC/precision-recall practice for skewed problems [3]. When the scored test set contains fewer than two classes, all metrics are returned as not-a-number by a guard. Five metrics are reported per run; the first three are primary.

PR-AUC (primary), prevalence-corrected. The area under the precision–recall curve (average precision) is the headline metric: it tracks performance on the minority class and is far more informative than ROC-AUC under extreme imbalance [4], [5]. A subtlety arises from test-set negative subsampling (Section 4.4 of the data chapter): to bound prediction cost, large datasets keep *all* positives but downsample negatives to a per-dataset cap (30,000; EU-CC is exempt and

scored in full). Subsampling negatives inflates the empirical positive-to-negative ratio and hence empirical precision. Because ROC counts (TPR, FPR) are invariant to random negative subsampling, precision is recomputed at the true full-split prevalence π ,

$$\text{precision}(t) = \frac{\pi \cdot \text{TPR}(t)}{\pi \cdot \text{TPR}(t) + (1 - \pi) \text{FPR}(t)},$$

and integrated as $\text{AP} = \sum_k (\text{recall}_k - \text{recall}_{k-1}) \text{precision}_k$ over score-sorted thresholds (`ap_at_prevalence`). At π equal to the observed prevalence this equals scikit-learn’s `average_precision_score` to machine precision, so unsubsampling datasets (EU-CC) are unaffected. This correction is what allows the dominant runtime speedup from negative subsampling without biasing the primary metric.

Recall@1%FPR and Recall@5%FPR (primary), conservative step values. Fraud systems operate at fixed, low false-positive budgets, so recall at 1% and 5% FPR are reported [6], [67]. Each is the **highest TPR among operating points with $\text{FPR} \leq \text{target}$** on the empirical ROC curve — a conservative step-function read, not a linear interpolation. Interpolation is deliberately avoided: an interpolated (FPR, TPR) pair between two ROC vertices does not correspond to any achievable decision threshold, so interpolating *overestimates* recall, markedly so when the positive class is tiny and the curve is coarse (e.g. EU-CC, with roughly 100 test frauds). The step value is the highest recall attainable at a real threshold that meets the FPR budget. Recall@FPR is invariant to negative subsampling; the discrete ROC grid imparts a small downward bias of order $1/N_{\text{neg, scored}}$, negligible at $N \approx 30,000$.

ROC-AUC and F1 (secondary). ROC-AUC (invariant to negative subsampling) is reported for comparability with prior fraud-detection work. F1 is computed at a fixed 0.5 probability threshold; because that threshold is sensitive to calibration and prevalence and is not prevalence-corrected, F1 is interpreted only as a coarse secondary indicator.

Aggregation. Results are aggregated across the three seeds (42, 123, 7) as mean $\pm$ standard deviation, except FiFAR, which uses a fixed published split and runs at a single seed (a point estimate with no spread). Only runs at the current pipeline version are analysed.

Status of RAP results. The C1 and C2 conditions are fully implemented and produce the baseline tables. The C3 and C4 conditions — the relevance effect and the fraud-ratio sweep that constitute RAP’s empirical contribution — are designed above but not yet implemented; their results, including the C3 single-class guard and the C4 duplication accounting, are pending Phase 3.

[Placeholder: Table 4.X — main C1 comparison, PR-AUC and Recall@1%FPR (mean $\pm$ sd) by model $\times$ dataset, FTMs vs GBDTs. Pending grid completion.]

[Placeholder: Table 4.Y — RAP ablation, PR-AUC for C1 / C2 / C3 / best-C4 per FTM $\times$ dataset. Pending C3/C4 implementation (Phase 3).]

[Placeholder: Figure 4.Z — C4 fraud-ratio sweep, PR-AUC vs target ratio per dataset, with duplication-threshold annotations. Pending C3/C4 implementation (Phase 3).]

5 Experimental Setup

This chapter specifies the experimental apparatus in enough detail to reproduce it: the datasets and their provenance, the splitting protocol, the preprocessing steps, the software implementation and reproducibility controls, the compute environment, and the full experiment grid. Every fact stated here is grounded in the code that produced the results — the dataset registry ([src/data/schema.py](#)), the splitter ([src/data/splitter.py](#)), the model wrappers ([src/models/](#)), and the experiment runner ([src/experiment/runner.py](#)). Where a design choice has a methodological consequence (for example, retaining time columns as features, or correcting precision–recall area under the curve for negative subsampling), the choice is stated explicitly so that downstream results can be interpreted correctly.

5.1 Datasets

The evaluation uses six fraud-detection datasets in the formal grid plus one development-only dataset. All seven are ingested into a single DuckDB database and described by a central registry; the registry records, per dataset, the label column, the categorical features, the columns to drop (entity identifiers and temporal keys), and the split protocol. Table 1 summarises the datasets. Counts are taken from the registry and the project fact sheet.

Table 2: Datasets used in the thesis. “Train fraud (~)” is the approximate number of fraudulent rows in the training split, which bounds context-construction feasibility under the balanced and oversampled conditions.

Dataset	Rows	Features	Fraud %	Train fraud (~)	In formal grid?	Seeds
eu_cc	285k	30	0.17%	394	Yes	42, 123, 7
cc_2025	500k	11	1.5%	—	Yes (de-generate)	42, 123, 7
banksim	595k	7	1.2%	5,760	Yes (grouped split)	42, 123, 7
paysim	6.3M	7	0.13%	6,570	Yes	42, 123, 7
fifar	603k	29	1.2%	5,705	Yes (fixed split)	1
baf	1M	30	1.1%	8,823	Yes	42, 123, 7
ai_banking	10k	6	28.4%	—	No (dev only)	—

The formal-grid datasets span a wide range of scale (285k to 6.3M rows) and feature dimensionality (7 to 30 features), but share the defining property of this thesis: heavy class imbalance, with fraud prevalence between roughly 0.13% and 1.5%. They divide cleanly by provenance. The European credit-card transactions benchmark ([eu_cc](#)) is the real, anonymised dataset released by

the Machine Learning Group at the Université Libre de Bruxelles; its 28 features are principal components (V1–V28) of the original transaction attributes, retained alongside `Time` and `Amount` for confidentiality [1]. It is the smallest and most extreme dataset, with only about 98 fraudulent transactions in the test split. BankSim (`banksim`) and PaySim (`paysim`) are agent-based simulators: BankSim is calibrated from aggregated payment data of a Spanish bank and emits labelled synthetic transactions over a six-month window [68], while PaySim simulates mobile-money transfers from one month of logs of a real African service, injecting fraud into the TRANSFER and CASH_OUT transaction types [42]. FiFAR (`fifar`) and the Bank Account Fraud suite (`baf`) are realistic, large-scale bank-account-opening fraud datasets released by Feedzai. FiFAR ships with a pre-defined train/test split and synthetic analyst predictions for human–AI collaboration research, of which only the transaction-level features and the binary label are used here [69]. BAF is a suite of six approximately one-million-row datasets engineered to expose imbalance, bias, and temporal distribution shift; this thesis uses the Base variant [7], [70].

Two datasets receive special treatment. The AI Banking 2025 dataset (`ai_banking`) [71] is flagged development-only in the registry: at 28.4% fraud it is nearly balanced and therefore unsuitable as an imbalance benchmark. It is used solely for developing and validating the RAP (Retrieval-Augmented Prediction) pipeline and the metric implementations, and it is excluded from all reported results. The Credit Card Fraud 2025 dataset (`cc_2025`) [72] is retained in the grid but flagged degenerate. Inspection shows that its fraud labels are statistically independent of every feature: the distributions of `Amount`, `Distance`, `Is_International`, and the categorical attributes are identical between fraudulent and legitimate transactions. As a consequence, all models — foundation models and gradient-boosted decision trees (GBDTs) alike — score a precision–recall area under the curve (PR-AUC) approximately equal to the fraud rate (≈ 0.015) and a receiver operating characteristic area under the curve (ROC-AUC) of approximately 0.50, i.e. random chance. The existing results are sufficient to document `cc_2025` in the thesis as a cautionary example of a poorly constructed synthetic benchmark; no further runs on it are queued, and its pre-existing result files are deliberately kept as that documentation.

5.2 Splitting protocol

Unless a dataset overrides it, the protocol is a seeded, stratified 80/20 train/test split using three seeds (42, 123, 7). Stratification preserves the fraud rate in both partitions; if the minority class is too rare for stratification, the splitter falls back to an unstratified random split. Two datasets override the default. BankSim uses a **grouped** split by `customer`: BankSim’s `customer` field is an entity identifier (roughly 4,112 customers across about 594k rows), and a random row-level split would place the same customer on both sides of the partition. The ordinal-encoded customer ID would then act as a memorisation key, leaking labels across the split and inflating measured performance. The grouped split (via `GroupShuffleSplit`) instead assigns each customer wholly to train or to test. Grouped splits are not label-stratified, but with thousands of groups the fraud rate stays close to the dataset rate. FiFAR uses its published, fixed pre-defined split; because the split is fixed, the seed has no effect and FiFAR is run with a single seed. This gives 16 dataset–seed units in the formal grid: five datasets at three seeds each, plus FiFAR at one seed.

Two methodological choices deserve emphasis. First, **time columns are deliberately kept as features** under the random split: `Time` in `eu_cc` and `step` in `banksim` and `paysim` are retained rather than dropped. This is a conscious decision — a random split can leak temporal structure between train and test — and the thesis treats the temporal-generalisation question separately as an ablation study rather than confounding it into the main grid. The natural vehicle for that ablation is BAF, whose published protocol splits by `month` (already excluded as a feature), giving a clean temporal-shift evaluation [7].

Second, **test-set negative subsampling** is applied to the large datasets to make the grid tractable. For `banksim`, `paysim`, `fifar`, and `baf`, every positive (fraudulent) test row is kept, and the negatives are randomly downsampled to a per-dataset cap of 30,000 (the registry’s `test_neg_cap`). The subsample is deterministic, drawn from an independent random stream (seed XOR 0x5CA1E) so it does not perturb the train-context sampling. `eu_cc` is always scored on the **full** test set (`test_neg_cap = None`), because with only ~98 test frauds, rare-event metrics under subsampling would be indefensible. This subsampling inflates the empirical positive-to-negative ratio and therefore empirical precision, so PR-AUC is **prevalence-corrected**: precision is recomputed at the true full-split fraud prevalence π using the relation $\text{precision}(t) = \pi \cdot \text{TPR}(t) / (\pi \cdot \text{TPR}(t) + (1-\pi) \cdot \text{FPR}(t))$, and the average precision is integrated over the score-sorted thresholds. Because the true positive rate (TPR) and false positive rate (FPR) of the empirical ROC curve are invariant to random negative subsampling, this correction recovers the PR-AUC that would have been observed on the full test set; at the observed prevalence it equals the standard average precision to machine precision. Recall-at-fixed-FPR and ROC-AUC are themselves invariant to negative subsampling and need no correction. The full-split counts (true prevalence) and the actually scored counts are both recorded per run so the correction is auditable. The metric details are developed in the evaluation chapter; the relevant point here is that subsampling is the dominant predict-time speedup (roughly 42× on PaySim) while leaving the reported metrics unbiased.

5.3 Preprocessing

The loader returns features as unscaled `float32` arrays and labels as `int32` 0/1 values. Categorical features (for example `type` in PaySim, or `payment_type` and `device_os` in BAF and FiFAR) are ordinal-encoded; unseen categories at test time map to a sentinel value of `-1`. Column names are lowercased and stripped, and the registry’s `drop` list removes entity identifiers and temporal keys (transaction IDs, customer/merchant names, and similar) that would otherwise leak. For FiFAR, additional analyst-prediction and testbed-metadata columns are dropped by name pattern, since these encode information unavailable at prediction time.

No scaling is applied for the GBDT baselines or for the random and stratified context-construction conditions, which are scale-invariant. Feature scaling is required only for the kNN (k-nearest-neighbours) retrieval index used by the retrieval-based conditions: there, a `StandardScaler` is fit on the training split only and applied to both partitions, and the scaled arrays are used **solely** to build the retrieval index. The context rows ultimately passed to the foundation models remain unscaled. One caveat applies to BankSim: its `customer` and `merchant` categoricals are ordinal-encoded entity IDs, over which Euclidean or cosine distance is meaningless, so retrieval relevance

on BankSim should be interpreted with care. The retrieval-based conditions are not yet implemented (see §5.6).

5.4 Implementation and reproducibility

The entire training, inference, and pipeline code is written in Python and managed with the `uv` package manager, with all versions pinned in a lockfile. The foundation models come from the `tabpfn` library at version 8.0.8 — covering TabPFN v2 [20], TabPFN 2.5 [21], [22], TabPFN 2.6 [36], and TabPFN v3 [37], [73] — together with TabICLv2 from the `tabicl` package [26]. The GBDT baselines are XGBoost [10] and CatBoost [11], described in the models chapter; both train on the **full** training set with no context limit, set `scale_pos_weight` automatically from the class counts, and use a held-out 10% validation slice for early stopping.

A single experiment run is fully specified by a configuration dictionary: dataset, model, condition, seed, and the retrieval-specific factors (batch size, retrieval pool size k , target fraud ratio, and retrieval metric) that apply only to the retrieval conditions, plus the foundation-model ensemble-pass count `n_estimators`. The **runner is idempotent**. It canonicalises the configuration (normalising irrelevant factors to a fixed sentinel so a stray value cannot fork an otherwise-identical run), serialises it deterministically, and takes the first 12 hexadecimal characters of the SHA-256 hash as the `run_id`. Output is written to `results/runs/{run_id}.parquet`; if that file already exists, the run is skipped. The runner is therefore safe to kill and restart. Crucially, the **pipeline version is part of the hashed configuration**: bumping it (currently version 4) invalidates all cached results, which is the mechanism used to force a clean rerun whenever model defaults or pipeline logic change. Library versions are recorded per run for provenance but are deliberately **not** hashed; the version-4 rerun keeps all reported results on one library version, avoiding a silent mix.

Two reproducibility caveats are worth stating up front. First, all reported foundation-model results use `n_estimators = 4` ensemble passes — a settled decision, set in the grid driver, to keep the full grid tractable on a single GPU; ensembling improves the stability of probabilistic predictions but costs predict time roughly linearly [66]. The runner’s own default for an ad-hoc single command is 8, so single runs invoked without an explicit `--n_estimators` are **not comparable** to grid results. Because `n_estimators` is in the configuration hash, the two settings produce distinct `run_ids` and distinct output rows, and the `n_estimators` column should always be checked before comparing. Second, all randomness flows through `numpy.random.default_rng(seed)`, so context sampling and the (independently seeded) test subsampling are fully reproducible.

5.5 Compute environment

All foundation-model inference runs on a single NVIDIA RTX 5070 (12 GB) with `device="cuda"`; XGBoost defaults to CPU. The single-GPU budget drives several of the tractability compromises already noted — `n_estimators = 4`, test-set negative subsampling, and a model-aware predict-time memory chunk that streams test rows through the model in fixed-size groups. The chunk size is purely a memory-management device: because test rows are conditionally independent given the fixed context under the random and stratified conditions, chunking groups rows without changing any output. Of these compromises, the predict-time test subsampling is the

dominant speedup. Because a concurrent GPU job may be in flight, runs are serialised through the idempotent driver rather than launched in parallel. Downstream analysis — results aggregation, tables, and figures — is done in R with Quarto and Typst; Python writes one Parquet row per run and R reads the result set directly, with no in-process bridge between the two languages.

5.6 The experiment grid

The grid is organised around five foundation models (TabPFN v2, TabPFN 2.5, TabPFN 2.6, TabPFN v3, and TabICLv2), two GBDT baselines, four context-construction conditions (C1 random subsample; C2 stratified balanced subsample; C3 kNN retrieval at natural ratio; C4 kNN retrieval with controlled fraud-ratio oversampling — the full RAP method), and the 16 dataset-seed units. GBDTs run under C1 only, since they train on the full dataset and the limited context window is an inherent foundation-model property under study. Table 2 enumerates the planned grid counts.

Block	Composition	Runs
FTM core	5 models $\times$ 4 conditions $\times$ 16 units	320
GBDT core	2 models $\times$ 1 condition (C1) $\times$ 16 units	32
C4 fraud-ratio sweep (net new)	5 models $\times$ 4 extra ratios $\times$ 16 units	320
Batch-size sweep	1 model $\times$ 4 batch sizes $\times$ 2 datasets $\times$ 3 seeds	24

At the time of writing the grid is being executed for the first two conditions. The retrieval-based conditions C3 and C4 — the core of the RAP contribution — are not yet implemented (the `src/rap/` package is a stub and the runner raises an error for those conditions), so all retrieval-effect and fraud-ratio-sweep results are pending. The TabPFN v3 weights additionally require a one-time license acceptance before they can be run; until that is in place the in-flight grid is effectively four foundation models, and the v3 cells are filled by a later idempotent rerun. Both gaps are confined to clearly demarcated blocks of Table 2.

[Placeholder: Table 3 — completed-run inventory by dataset $\times$ model $\times$ condition (pipeline version 4). Pending grid completion.]

6 Results, Discussion and Conclusion

This chapter reports the empirical results of the thesis, discusses their interpretation, and concludes with limitations and future work. Because the experimental grid is still being collected at the time of writing, the chapter is organised so that the analysis plan — what each table and figure will show, how results are aggregated, and which research question (RQ) each addresses — is fully specified now, with numeric cells left as clearly-labelled placeholders to be filled as runs complete.

6.1 Status of the experimental grid

The grid is partially collected. All results reported in the thesis are filtered to pipeline version 4 (the constant `_PIPELINE_VERSION = 4` in the run orchestrator), under which the test set is negative-subsampled on large datasets — all positives are kept, negatives are capped at 30,000 per dataset — and precision-recall area under the curve (PR-AUC) is prevalence-corrected so that it reflects the true population fraud rate rather than the inflated positive-to-negative ratio of the scored subset. Earlier pipeline-v2 and -v3 parquet files are stale and excluded from analysis, with the sole exception of the degenerate `cc_2025` dataset (discussed in §Discussion), whose pre-v4 files are retained deliberately as documentation.

Two facts about the current state must be stated plainly so that the results below are not over-read:

1. **Only the random-subsampling baseline (condition C1) and the stratified 50/50 baseline (condition C2) have been collected under pipeline v4, and only on a subset of datasets.** As of writing, v4 runs exist for BankSim (most cells), EU credit-card (EU-CC), and FiFAR, with PaySim and BAF reruns still queued. The headline comparison datasets are therefore not yet complete.
2. **The retrieval arms — C3 (k-nearest-neighbour, kNN, retrieval at the natural fraud ratio) and C4 (kNN retrieval plus fraud-ratio oversampling, i.e. Retrieval-Augmented Prediction, RAP) — are not yet implemented.** The `src/rap/` package currently contains only a stub, and the runner raises `NotImplementedError` for these conditions. **All RAP-specific results — the relevance effect (C3 versus C1) and the fraud-ratio sweep (C4) — are therefore pending, and no headline RAP numbers are available in this draft.**

Consequently this chapter presents the C1/C2 baseline analysis plan with placeholders for the still-incomplete cells, and the C3/C4 analyses entirely as a pre-registered plan. This separation is deliberate: stating the analysis design before the RAP results are in fixes the comparisons and aggregations in advance and guards against post-hoc metric selection, which is especially important under heavy class imbalance where metric choice can flatter a method [3].

Aggregation conventions, applied uniformly across all tables and figures below: each cell is the mean $\pm$ standard deviation over the dataset’s seeds (42, 123, 7). FiFAR is reported as a single point with no error bar, because it uses a fixed published train/test split and is run once; this is noted wherever FiFAR appears. The primary metrics are PR-AUC (prevalence-corrected on subsampled datasets), Recall at 1% false-positive rate (FPR) and Recall at 5% FPR; ROC-AUC and F1 are secondary. Recall@FPR uses the conservative step value of the empirical ROC curve — the highest true-positive rate attainable at a real decision threshold whose FPR does not exceed the budget

— rather than ROC interpolation, which would overestimate achievable recall on the coarse ROC curves produced by datasets with very few positives such as EU-CC.

6.2 RQ1 — FTMs versus GBDTs under random subsampling (C1)

RQ1 asks whether in-context-learning (ICL) tabular foundation models (FTMs), restricted to their context window and fed a random subsample of the training data, can match gradient-boosted decision trees (GBDTs) trained on the full training set. This is the central baseline comparison: the GBDTs (XGBoost and CatBoost) are deliberately given no context limit, because the limited context window is an inherent property of ICL FTMs and is the very constraint the thesis interrogates [10], [11], [13]. The FTMs evaluated are TabPFN v2, TabPFN 2.5, TabPFN 2.6, TabPFN v3, and TabICLv2 [20], [21], [26]. TabPFN v3 is currently excluded from the live grid pending one-time license acceptance, so the grid in flight is effectively four FTMs plus two GBDTs.

The main RQ1 table reports, for every model $\times$ dataset cell, PR-AUC and Recall@1%FPR as mean $\pm$ sd over seeds (FiFAR single point).

[Placeholder: Table 1 — RQ1 main C1 comparison. Rows: XGBoost, CatBoost, TabPFN v2, TabPFN 2.5, TabPFN 2.6, TabPFN v3, TabICLv2. Columns grouped by dataset (EU-CC, BankSim, PaySim, FiFAR, BAF); two metrics per dataset (PR-AUC, Recall@1%FPR), each mean $\pm$ sd over 3 seeds (FiFAR: single value). Pending grid completion (C1; PaySim/BAF reruns and TabPFN v3 cells outstanding).]

The expected reading of this table is conditional. Prior work shows FTMs matching or beating tuned GBDTs on small and medium balanced tables [20], but those benchmarks are not heavily imbalanced and the FTMs there see the whole (small) dataset. Here, on datasets with hundreds of thousands to millions of rows and fraud rates near 0.1–1.2%, a random context subsample to 10,000 rows (TabPFN v2) or ~48–50,000 rows (the larger-context models) will contain only a handful of fraud examples — at EU-CC’s 0.17% rate, a 10,000-row context contains roughly 17 frauds in expectation. The plausible outcome is that GBDTs, which use all training fraud, retain an advantage on PR-AUC, and that the gap widens with dataset size and rarity. RQ1 establishes the size of that gap; the RAP arms (RQ3/RQ4) test whether retrieval can close it.

6.3 RQ2 — does random subsampling degrade at scale? (C1 across size tiers)

RQ2 isolates the scaling behaviour of the C1 baseline. Holding the random-subsampling strategy fixed, it asks whether FTM performance degrades, relative to the full-data GBDTs, as the dataset grows and the fraction of training fraud captured by a fixed-size random context shrinks. This is the mechanism that motivates RAP: under heavy imbalance, a larger pool means a random context window is flooded with ever more irrelevant legitimate transactions. The degradation of a fixed random context as data grows has been reported for TabPFN [31], [32]; RQ2 re-confirms it for the modern FTM line specifically under fraud-scale imbalance and with prevalence-corrected PR-AUC.

[Placeholder: Figure 1 — RQ2 C1 PR-AUC versus $\log_{10}(\text{dataset rows})$. One line per FTM and one per GBDT (mean over seeds, shaded $\pm sd$ band; FiFAR single marker). x-axis: $\log$ dataset size (EU-CC $\approx 285k$, BankSim $\approx 595k$, FiFAR $\approx 603k$, BAF $\approx 1M$, PaySim $\approx 6.3M$). Expected: GBDT lines roughly flat or rising; FTM lines flat-to-declining, widening the GBDT–FTM gap at the high end. Pending grid completion.]

The figure plots PR-AUC against the base-10 logarithm of dataset row count, with one trace per model. If random subsampling degrades at scale, the FTM traces should fall, or at least fail to track the GBDT traces, as size increases — most visibly between BAF ($\sim 1M$ rows) and PaySim ($\sim 6.3M$ rows). A flat GBDT trace would confirm that the effect is a context-window artefact of the FTMs rather than an intrinsic difficulty of the larger datasets. Dataset size is a coarse proxy here; the operative variable is the expected number of fraud examples surviving in a fixed-size random context, which the discussion will report alongside the figure.

6.4 RQ3 — does retrieval improve over random subsampling? (C1 $\rightarrow$ C3 $\rightarrow$ C4 ablation)

RQ3 asks whether building the context by retrieval, rather than by random subsampling, improves detection under heavy imbalance. It is largely *confirmatory*: LoCalPFN’s frozen TabPFN-kNN and TabDPT already show retrieval beating a random context for tabular ICL [31], [32], so RQ3 re-tests that benefit on modern FTMs under fraud-scale imbalance — and against the competing evidence that random context selection is a strong baseline [33]. The thesis’s novel question is RQ4. The design decomposes the answer into two interpretable effects via the C1 $\rightarrow$ C3 $\rightarrow$ C4 chain:

- **C3 versus C1 isolates the relevance effect.** C3 retrieves, for each test batch, the top- k nearest training rows in StandardScaler-scaled feature space (scaler fit on train only; context rows themselves remain unscaled when passed to the FTM) and keeps the *natural* fraud ratio of that neighbourhood. Any improvement over C1 is attributable to relevance alone — a structurally similar context — with the ratio held fixed.
- **C4 versus C3 isolates the ratio effect.** C4 takes the same retrieval and then enforces a target fraud ratio swept over {5%, 10%, 20%, 30%, 50%}, oversampling fraud as needed. Improvement of the best C4 ratio over C3 is attributable to ratio tuning, with retrieval held fixed.

This decomposition is the interpretive contribution of the design: it separates *which rows* are in the context (relevance) from *how balanced* the context is (ratio), two confounded levers in any single rebalanced-context method. Framing the in-context set as a controllable test-time lever follows the FTM-robustness literature, which manipulates the context adversarially [29]; RAP is a benign, imbalance-motivated instance of the same mechanism. Conceptually, RAP rebalances at inference time what SMOTE and focal loss rebalance at training time [2], [8], [9], with the advantage that it requires no retraining of the frozen FTM. The only prior method to combine retrieval with imbalance handling for tabular ICL, AWARE, does so by *learning* a task-aligned retrieval embedding rather than by controlling the context ratio [62]; RAP keeps both the model and the retriever fixed and intervenes only on the label composition.

[Placeholder: Table 2 / Figure 2 — RQ3 ablation per dataset. Per dataset and FTM, four bars or columns: C1 (random), C2 (stratified 50/50), C3 (kNN, natural ratio), C4-best (kNN + best swept fraud ratio). Metric: PR-AUC mean±sd over seeds (FiFAR single point). Annotate the C3–C1 gap (relevance) and the C4best–C3 gap (ratio). Pending C3/C4 implementation (Phase 3).]

C2 (stratified 50/50 over the whole training set, no retrieval) is included in this comparison as a second baseline: it rebalances the ratio without relevance, so the contrast C2-versus-C4 indirectly cross-checks whether retrieval adds anything beyond rebalancing. Under the imbalance hypothesis, the expected ordering is $C1 < C2 \approx C3 < C4\text{-best}$, but the relative size of the relevance and ratio effects is exactly what the experiment is designed to measure and is not assumed.

A required implementation detail, noted here because it conditions the results: C3 must guard against fraud-free retrieved contexts, since a single-class context label vector breaks the positive-class probability extraction. The planned guard supplements an all-legitimate neighbourhood with fraud drawn from the global training pool.

6.5 RQ4 — the optimal context fraud ratio (C4 sweep)

RQ4 is the thesis’s primary and most novel question: what fraud ratio in the retrieved context maximises detection, and is the optimum stable across datasets and models? No retrieval-augmented tabular method controls the in-context label ratio, and the closest evidence is discouraging — in language-model ICL, rebalancing the demonstration class ratio (including a 50/50 split) was found ineffective or harmful [34], and on balanced tabular data smart context selection barely beats random [33]. Whether the heavy-imbalance fraud regime behaves differently is an open question, so a null result or an interior optimum is each an informative finding. This sweep underlies the “C4-best” column of the RQ3 ablation.

[Placeholder: Figure 3 — RQ4 PR-AUC by context fraud ratio. x-axis: target fraud ratio {0.05, 0.10, 0.20, 0.30, 0.50}; y-axis: PR-AUC (mean±sd over seeds; FiFAR single point). One panel per dataset, one line per FTM. Annotate cells whose duplication factor (context_fraud_n / context_fraud_unique) exceeds the stated threshold. Pending C4 implementation.]

The natural hypothesis is an interior optimum — too little fraud reproduces the C1 problem, while a fully balanced 50/50 context over-represents fraud relative to deployment and may hurt calibration and precision — but the curve shape is an empirical question. The figure will also expose the cost of oversampling: at the larger context sizes (~48–50k) no dataset reaches the 20% ratio or above using unique fraud rows alone, so high-ratio cells are achieved by duplicating fraud examples. The duplication factor is logged per run (the ratio of fraud slots to distinct fraud rows), and cells above a stated duplication threshold are annotated or excluded at analysis time rather than silently reported.

6.6 Discussion

The degenerate cc_2025 dataset. The Credit Card Fraud 2025 synthetic dataset is retained in the thesis not as a result but as a cautionary example. Its fraud labels are statistically independent of every feature — amount, distance, international flag and so on are identically distributed for fraud

and non-fraud — so no model can do better than chance. Every model scores PR-AUC ≈ 0.015 (the fraud rate) and ROC-AUC ≈ 0.50 , exactly as theory predicts for label-independent features. This is a useful negative control in two ways. First, it validates the evaluation pipeline: a prevalence-corrected PR-AUC that lands on the base rate, and a ROC-AUC of 0.5, are the correct outputs for an uninformative dataset, so the metrics are behaving as intended. Second, it is a concrete warning about synthetic fraud benchmarks: a dataset can look realistic (plausible columns, plausible imbalance) while encoding no learnable signal at all. Because no further `cc_2025` runs add information, the dataset is excluded from the active grid and documented only as an example of a poorly-constructed synthetic benchmark, a concern echoed in the synthetic-data literature on realism [56], [59].

Library-version provenance. Every run records the resolved `tabpfn` and `tabicl` library versions (currently `tabpfn` 8.0.8). These versions are recorded for provenance but are deliberately *not* part of the configuration hash. The risk this creates — a silent mixture of results produced by different library versions — is managed by bumping the pipeline version and rerunning the whole grid on a single library version, so that all reported v4 results share one software stack. This is a deliberate methodological choice: provenance fields make the version auditable per row, while the full-rerun discipline keeps the comparison clean.

Interpretive value of the relevance-versus-ratio decomposition. Most context-rebalancing or scaling approaches for TabPFN change retrieval and balance together [25], leaving it unclear which lever drives any gain. The $C1 \rightarrow C3 \rightarrow C4$ chain is designed so that the two effects can be read off separately. If, as expected, the experiments show that ratio tuning contributes most of the benefit, that has a practical implication: a cheaper stratified context (C2) might recover much of RAP’s gain without per-batch retrieval. If instead relevance dominates, retrieval is essential and the method cannot be shortcut. Either outcome is informative, and the value of the design is that it can tell the two apart rather than reporting a single conflated number. This positions RAP as a test-time, training-free intervention complementary to model-side advances such as better priors [24] and to the broader finding that, under extreme imbalance, the training objective can matter more than architecture [35]. It also separates RAP from the two closest imbalance-aware retrieval methods, both of which take the opposite, training-heavy route: AWARE learns a task-aligned retrieval embedding [62] and IM-Context rebalances a fixed-mixture context for regression [65], whereas RAP intervenes only on the in-context fraud ratio of a frozen model. Should the ratio sweep show no benefit, that too is a contribution: it would extend the negative ICL-rebalancing result [34] from language to heavily imbalanced tabular data.

6.7 Conclusion, Limitations, and Future Work

6.7.1 Limitations

Several limitations bound the interpretation of the results and the method.

With-replacement oversampling duplication. Filling the fraud quota in C4 duplicates fraud rows when unique training fraud is exhausted. Oversampling is the method, so duplication is not capped — but duplication-heavy cells (notably EU-CC, with only ~394 training frauds, and all models at the 48–50k context for ratios $\geq 20\%$) carry little new information per duplicate. These

cells are annotated or excluded via a stated duplication threshold at analysis time, not removed from the grid, so the protocol stays uniform across datasets while the reporting stays honest. This design also inherits a known weakness: SMOTE’s original analysis found that with-replacement duplication of minority examples does not significantly improve minority-class recognition relative to feature-space interpolation [8]. RAP duplicates rather than synthesises because a frozen FTM consumes real rows and cannot ingest interpolated points without an encoder, so every context row stays a genuine, audit-traceable transaction; whether duplication suffices or interpolation-style synthesis would help is left to future work.

kNN over ordinal-encoded entity IDs. Retrieval (C3/C4) computes distances in scaled feature space, but some categorical features are ordinal-encoded entity identifiers — most clearly BankSim’s `customer` and `merchant`. Euclidean or cosine distance over an arbitrary integer ID coding is meaningless, so retrieval relevance is questionable on such datasets and is flagged accordingly. This is also why BankSim uses a grouped-by-customer split: a row-level split would let the encoded customer ID act as a label-leaking memorisation key [68].

Single-GPU scale. The entire grid runs on one NVIDIA 5070, forcing tractability compromises: a fixed four-pass FTM ensemble, test-set negative subsampling, and predict chunking. Negative subsampling and chunking are result-neutral by construction (Recall@FPR and ROC-AUC are invariant to negative subsampling, PR-AUC is prevalence-corrected, and test rows are independent given a fixed context), but the ensemble-size choice is not.

The `n_estimators = 4` trade-off. All reported FTM grid results use four ensemble passes, half the library default of eight, to keep the full grid tractable on one GPU. Reducing the ensemble size speeds prediction roughly linearly but changes outputs, and it is part of the configuration hash. Single ad-hoc command-line runs default to eight passes and are therefore *not* comparable to grid runs; comparisons must always check the `n_estimators` column [66].

Random rather than temporal split. The main grid uses random stratified splits and deliberately keeps time columns (`Time` in EU-CC, `step` in BankSim and PaySim) as features. This can leak temporal structure and is stated explicitly as a methods choice. The temporal-distribution-shift question is deferred to an ablation rather than baked into the main comparison [38], [58].

6.7.2 Future work

The immediate priority is to finish the grid: implement the RAP modules so that C3 and C4 run, complete the PaySim and BAF v4 reruns, and add the TabPFN v3 cells once the license is accepted. Beyond completing the planned analyses (the batch-size sweep and the C4 fraud-ratio sweep), the natural extension is the temporal-split ablation using BAF’s published month-based protocol, in which `month` is already excluded as a feature, to test whether RAP’s benefit survives distribution shift [7], [70]. A further direction is comparing retrieval metrics (cosine versus Euclidean) and principled handling of entity-ID features so that retrieval is meaningful on datasets like BankSim.

6.7.3 Restatement of the contribution

This thesis introduces Retrieval-Augmented Prediction, a test-time context-construction strategy for in-context-learning tabular foundation models under heavy class imbalance. Rather than randomly subsampling the training set into the FTM’s limited context window — which, under

fraud-rate imbalance, floods the window with irrelevant legitimate transactions — RAP retrieves a structurally similar, label-balanced context around each test batch using kNN retrieval and controlled fraud-ratio oversampling. The method is training-free and operates on a frozen FTM, and its evaluation is designed to decompose the benefit into a relevance effect and a ratio effect. The baseline grid is in progress and the RAP arms are pending implementation, but the experimental design, metrics, and analysis plan presented here fix the comparison in advance so that the headline results, once collected, answer each research question directly.

References

Bibliography

- [1] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, “Calibrating Probability with Undersampling for Unbalanced Classification,” in *2015 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2015, pp. 159–166. doi: 10.1109/SSCI.2015.33.
- [2] H. He and E. A. Garcia, “Learning from Imbalanced Data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.
- [3] T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.
- [4] J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, New York, NY, USA: ACM, 2006, pp. 233–240. doi: 10.1145/1143844.1143874.
- [5] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLOS ONE*, vol. 10, no. 3, p. e118432, 2015, doi: 10.1371/journal.pone.0118432.
- [6] Y.-A. Le Borgne, W. Siblini, B. Lebichot, and G. Bontempi, *Reproducible Machine Learning for Credit Card Fraud Detection – Practical Handbook*. Université Libre de Bruxelles, 2022. [Online]. Available: <https://github.com/Fraud-Detection-Handbook/fraud-detection-handbook>
- [7] S. Jesus *et al.*, “Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation.” [Online]. Available: <https://arxiv.org/abs/2211.13358>
- [8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002, doi: 10.5555/1622407.1622416.
- [9] T.-Y. Lin, P. Goyal, R. B. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in *International Conference on Computer Vision (ICCV)*, 2017. doi: 10.1109/ICCV.2017.324.
- [10] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, ACM, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.
- [11] A. V. Dorogush, V. Ershov, and A. Gulin, “CatBoost: gradient boosting with categorical features support,” *arXiv preprint*, 2018, [Online]. Available: <https://arxiv.org/abs/1810.11363>
- [12] G. Ke *et al.*, “LightGBM: A Fast, Distributed, High Performance Gradient Boosting Decision Tree Framework,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: <https://papers.nips.cc/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf>

- [13] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks track*, 2022. [Online]. Available: <https://arxiv.org/abs/2207.08815>
- [14] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [15] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [16] S. Müller, N. Hollmann, S. P. Arango, J. Grabocka, and F. Hutter, “Transformers Can Do Bayesian Inference,” in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: <https://openreview.net/forum?id=KSugKcbNf9>
- [17] Samuel Gabriel and contributors, “PFNs: Our maintained PFN repository. Come here to train SOTA PFNs..” 2026.
- [18] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter, “TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://openreview.net/forum?id=cp5PvcI6w8_
- [19] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter, “TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second.” [Online]. Available: <https://arxiv.org/abs/2207.01848>
- [20] N. Hollmann *et al.*, “Accurate predictions on small data with a tabular foundation model,” *Nature*, vol. 637, pp. 319–326, 2025, doi: 10.1038/s41586-024-08328-6.
- [21] L. Grinsztajn *et al.*, “TabPFN-2.5: Advancing the State of the Art in Tabular Foundation Models.” [Online]. Available: <https://arxiv.org/abs/2511.08667>
- [22] Prior Labs Team, “TabPFN-2.5 Model Report (web page).” 2025.
- [23] H.-J. Ye, S.-Y. Liu, and W.-L. Chao, “A Closer Look at TabPFN v2: Understanding Its Strengths and Extending Its Capabilities.” [Online]. Available: <https://arxiv.org/abs/2502.17361>
- [24] A. Garg, M. Ali, N. Hollmann, L. Purucker, S. Müller, and F. Hutter, “Real-TabPFN: Improving Tabular Foundation Models via Continued Pre-training With Real-World Data.” [Online]. Available: <https://arxiv.org/abs/2507.03971>
- [25] S.-Y. Liu and H.-J. Ye, “TabPFN Unleashed: A Scalable and Effective Solution to Tabular Classification Problems.” [Online]. Available: <https://arxiv.org/abs/2502.02527>
- [26] J. Qu, D. Holzmüller, G. Varoquaux, and M. Le Morvan, “TabICLv2: A better, faster, scalable, and open tabular foundation model.” [Online]. Available: <https://arxiv.org/abs/2602.11139>

- [27] R. Olusegun and B. Yang, “Improved Ethereum Fraud Detection Mechanism with Explainable Tabular Transformer Model,” in *2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2024. doi: 10.1109/TPS-ISA62245.2024.00017.
- [28] Prior Labs, “How TabPFN is Transforming Financial Risk Management (case study with Taktile).” 2025.
- [29] M. Djilani *et al.*, “On the Robustness of Tabular Foundation Models: Test-Time Attacks and In-Context Defenses.” [Online]. Available: <https://arxiv.org/abs/2506.02978>
- [30] Y. Gorishniy, I. Rubachev, N. Kartashev, D. Shlenskii, A. Kotelnikov, and A. Babenko, “TabR: Tabular Deep Learning Meets Nearest Neighbors,” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://openreview.net/forum?id=rhgIgTSSxW>
- [31] J. Ma *et al.*, “TabDPT: Scaling Tabular Foundation Models on Real Data.” [Online]. Available: <https://arxiv.org/abs/2410.18164>
- [32] V. Thomas *et al.*, “Retrieval & Fine-Tuning for In-Context Tabular Models,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. [Online]. Available: <https://openreview.net/forum?id=337dHOexCM>
- [33] B. Feuer, C. Hegde, and N. Cohen, “Scaling TabPFN: Sketching and Feature Selection for Tabular Prior-Data Fitted Networks,” in *2nd Table Representation Learning Workshop, NeurIPS 2023*, 2023. [Online]. Available: <https://arxiv.org/abs/2311.10609>
- [34] H. Gao, F. Zhang, H. Zeng, D. Meng, B. Jing, and H. Wei, “Exploring Imbalanced Annotations for Effective In-Context Learning.” [Online]. Available: <https://arxiv.org/abs/2502.04037>
- [35] W. Sun, Q. Shen, Y. Gao, Q. Mao, T. Qi, and S. Xu, “Objective over Architecture: Fraud Detection Under Extreme Imbalance in Bank Account Opening,” *Computation*, vol. 13, no. 12, p. 290, 2025, doi: 10.3390/computation13120290.
- [36] Prior Labs, “TabPFN-2.6 Model Card.” [Online]. Available: https://huggingface.co/Prior-Labs/tabpfn_2_6
- [37] L. Grinsztajn *et al.*, “TabPFN-3: Technical Report.” [Online]. Available: <https://arxiv.org/abs/2605.13986>
- [38] K. Helli, D. Schnurr, N. Hollmann, S. Müller, and F. Hutter, “Drift-Resilient TabPFN: In-Context Learning Temporal Distribution Shifts on Tabular Data.” [Online]. Available: <https://openreview.net/pdf/2c60a32c113b44bad1c212ba10e205aa4ad4b09f.pdf>
- [39] L. Xu, Y. Zhang, Q. Gan, M. Wang, and D. Wipf, “No Need to Train Your RDB Foundation Model.” [Online]. Available: <https://arxiv.org/abs/2602.13697>
- [40] N. Erickson *et al.*, “TabArena: A Living Benchmark for Machine Learning on Tabular Data.” 2025.

- [41] J.-P. Jiang, S.-Y. Liu, H.-R. Cai, Q.-L. Zhou, and H.-J. Ye, “Representation Learning for Tabular Data: A Comprehensive Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026, doi: 10.1109/TPAMI.2026.3657217.
- [42] E. A. Lopez, A. Elmir, and S. Axelsson, “PaySim: A Financial Mobile Money Simulator for Fraud Detection,” in *European Modeling and Simulation Symposium (EMSS)*, 2016, pp. 249–255.
- [43] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, “What Makes Good In-Context Examples for GPT-3?,” in *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, Dublin, Ireland and Online: Association for Computational Linguistics, May 2022, pp. 100–114. doi: 10.18653/v1/2022.deelio-1.10.
- [44] O. Rubin, J. Herzig, and J. Berant, “Learning To Retrieve Prompts for In-Context Learning,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Seattle, United States: Association for Computational Linguistics, July 2022, pp. 2655–2671. doi: 10.18653/v1/2022.naacl-main.191.
- [45] T. M. Cover and P. E. Hart, “Nearest Neighbor Pattern Classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967, doi: 10.1109/TIT.1967.1053964.
- [46] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. in Springer Series in Statistics. New York, NY: Springer, 2009. doi: 10.1007/978-0-387-84858-7.
- [47] Prior Labs, “PriorLabs/TabPFN: TabPFN Foundation Model for Tabular Data.” 2026.
- [48] Prior Labs, “PriorLabs/tabpfm-extensions.” 2026.
- [49] Prior Labs, “TabPFN Documentation: Anomaly Detection.” 2026.
- [50] P. Wang and S. Li, “Multi-Agent Debate: A Unified Agentic Framework for Tabular Anomaly Detection.” [Online]. Available: <https://arxiv.org/abs/2602.14251>
- [51] P. Baghdadi, S. Korukoglu, M. A. Bilici, and A. Onan, “The Potential of Energy-Based RBM and xLSTM for Real-Time Predictive Analytics in Credit Card Fraud Detection,” *Journal of Data Analysis and Information Processing*, vol. 13, no. 1, 2025, doi: 10.4236/jdaip.2025.131005.
- [52] M. A. Ghalwash, S. M. Abdelrazek, N. H. Eladawi, and others, “Enhancing credit card fraud detection using DBSCAN-augmented disjunctive voting ensemble,” *Scientific Reports*, vol. 15, p. 39754, 2025, doi: 10.1038/s41598-025-22960-w.
- [53] T. Albalawi and S. Dardouri, “Enhancing credit card fraud detection using traditional and deep learning models with class imbalance mitigation,” *Frontiers in Artificial Intelligence*, 2025, doi: 10.3389/frai.2025.1643292.
- [54] X. Li *et al.*, “An Intelligent Financial Fraud Detection Support System Based on Three-Level Relationship Penetration,” *Mathematics*, vol. 12, no. 14, p. 2195, 2024, doi: 10.3390/math12142195.

- [55] A. Patil, S. Mahajan, J. Menpara, S. Wagle, P. Pareek, and K. Kotecha, “Enhancing fraud detection in banking by integration of graph databases with machine learning,” *MethodsX*, 2024, doi: 10.1016/j.mex.2024.102683.
- [56] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, “Realistic Synthetic Financial Transactions for Anti-Money Laundering Models.” [Online]. Available: <https://arxiv.org/abs/2306.16424>
- [57] F. Wójcik, “Money Laundering Detection with Multi-Aggregation Custom Edge GIN,” *Journal of Data Science*, 2025, doi: 10.6339/25-JDS1190.
- [58] Y. Kim, Y. Lee, M. Choe, S. Oh, and Y. Lee, “Temporal Graph Networks for Graph Anomaly Detection in Financial Networks.” [Online]. Available: <https://arxiv.org/abs/2404.00060>
- [59] S. Huda, E. Foo, Z. Jadidi, M. H. Newton, and A. Sattar, “AMLNet: A Knowledge-Based Multi-Agent Framework to Generate and Detect Realistic Money Laundering Transactions.” [Online]. Available: <https://arxiv.org/abs/2509.11595>
- [60] T. Okeke, H. Dhindsa, A. Bejj, and D. Fenton, “Detecting Mule Account Fraud with Federated Learning.” 2025.
- [61] P. Alumona, O. Lawal, M. O. Ikhifa, D. O. Agbeso, O. Awele, and D. Olukoya, “Fraud Detection in Financial Transactions Using Machine Learning: Insights from the PaySim Mobile Money Dataset.” [Online]. Available: <https://zenodo.org/records/17875257>
- [62] M.-K. Pham *et al.*, “Retrieval-aligned Tabular Foundation Models Enable Robust Clinical Risk Prediction in Electronic Health Records Under Real-world Constraints.” [Online]. Available: <https://arxiv.org/abs/2604.01841>
- [63] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, “Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds., Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 8086–8098. doi: 10.18653/v1/2022.acl-long.556.
- [64] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, “Calibrate Before Use: Improving Few-Shot Performance of Language Models,” in *Proceedings of the 38th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 12697–12706. [Online]. Available: <https://proceedings.mlr.press/v139/zhao21c.html>
- [65] I. Nejjar, F. Ahmed, and O. Fink, “IM-Context: In-Context Learning for Imbalanced Regression Tasks.” [Online]. Available: <https://arxiv.org/abs/2405.18202>
- [66] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.04599>
- [67] D. K. McClish, “Analyzing a Portion of the ROC Curve,” *Medical Decision Making*, vol. 9, no. 3, pp. 190–195, 1989, doi: 10.1177/0272989X8900900307.

- [68] E. A. López-Rojas and S. Axelsson, “BankSim: A Bank Payments Simulator for Fraud Detection Research,” in *Proceedings of the 26th European Modeling and Simulation Symposium (EMSS 2014)*, 2014, pp. 144–152.
- [69] J. V. Alves *et al.*, “FiFAR: A Fraud Detection Dataset for Learning to Defer.” [Online]. Available: <https://arxiv.org/abs/2312.13218>
- [70] Feedzai, “feedzai/bank-account-fraud: Bank Account Fraud (BAF) Tabular Dataset Suite.” 2026.
- [71] M. Talha, “AI-Powered Banking Fraud Detection Dataset (2025).” [Online]. Available: <https://www.kaggle.com/datasets/mdtalhask/ai-powered-banking-fraud-detection-dataset-2025>
- [72] P. Rajak, “Credit Card Fraud 2025.” [Online]. Available: <https://www.kaggle.com/datasets/prince7489/credit-card-fraud-2025>
- [73] Prior Labs, “TabPFN v8.0.0 Release Notes.” [Online]. Available: <https://github.com/PriorLabs/TabPFN/releases/tag/v8.0.0>